

The Mist

A 2D RPG puzzle adventure,
save the animals from the
corruption spreading throughout
the lands.



Brona Keevers-Roux
Computer science – Game development
Supervisor: Brendan Lyng
Target platform: Windows

2. Concept Summary

This is a 2D top-down RPG with turn-based combat, exploration and puzzle solving. The player sets out on a quest to save the world from a dark mist that is spreading throughout the land, corrupting the wildlife and making them all highly aggressive. The player must find the source of the corruption, defeat the evil rat and destroy the cursed machine. To reach the device the player must fight their way through the forest, exploring temples and unlocking elemental powers that will aid them on their journey.

Players will control both the main character and a pet hamster companion who can access small areas in the form of side scrolling platformer sections to collect items or unlock mechanisms and allow the player to progress onwards.

Genres

2D Top-down RPG

Puzzle adventure

Turn-based

Target Audience

The target audience is players who enjoy exploration driven games and problem solving. The turn-based battle system will also appeal to players who prefer a slower paced combat. The procedurally generated temples and arcade mode should attract those who enjoy dungeon crawler type games.

Platform

The game will be made for PC, supporting keyboard and mouse controls.

Core Fantasy / Player Experience Goal

The goal of *The Mist* is to create a sense of adventure and encourage the player to explore. The game should be challenging enough to give the player a sense of achievement when completing puzzles or defeating tough enemies. The player should feel emotionally connected to their pet hamster. The player experience is centred on exploration and mastery, as the player progresses through the game they will grow in strength and skill. The gameplay aims to create a state of flow with challenges increasing in difficulty to match the players skill level.

3. Unique Selling Points

- **Dual gameplay perspectives**

The Mist blends two perspectives, the main game which is played from a top-down perspective and side view platforming sections where the player is controlling their pet hamster companion.

- **Procedurally generated temples**

The game provides replay-ability with its procedurally generated temples making every playthrough unique.

- **Elemental powers**

The player can unlock elemental powers which can be used to solve puzzles and unlock paths needed to progress the story.

- **Turn-based combat with reactive AI**

The turn-based combat system relies on adaptative AI to analyse the players moves and react accordingly to give the player a good challenge.

4. Core Gameplay Loop

The core gameplay loop focuses on exploration, combat and progression. Players must explore, solve puzzles, fight battles and complete temples to progress.

Main Gameplay Loop

Explore -> Battle -> Solve -> Unlock -> Progress

1. Explore

The player begins by exploring the world, travelling through the corrupted forest. Through exploration the player will discover temples, enemies, lootable chests and puzzles. The hamster companion can be sent into small gaps to collect loot and special items such as keys the player will need to continue forward. The mini map will help the player to thoroughly explore by showing small pathways that the player may have missed.

2. Battle

While exploring the player will encounter corrupted wildlife and enemies. Turn based combat ensues where players will choose their actions strategically to defeat the enemies. Enemy AI reacts to player choices. Winning battles will earn the player experience and loot.

3. Solve

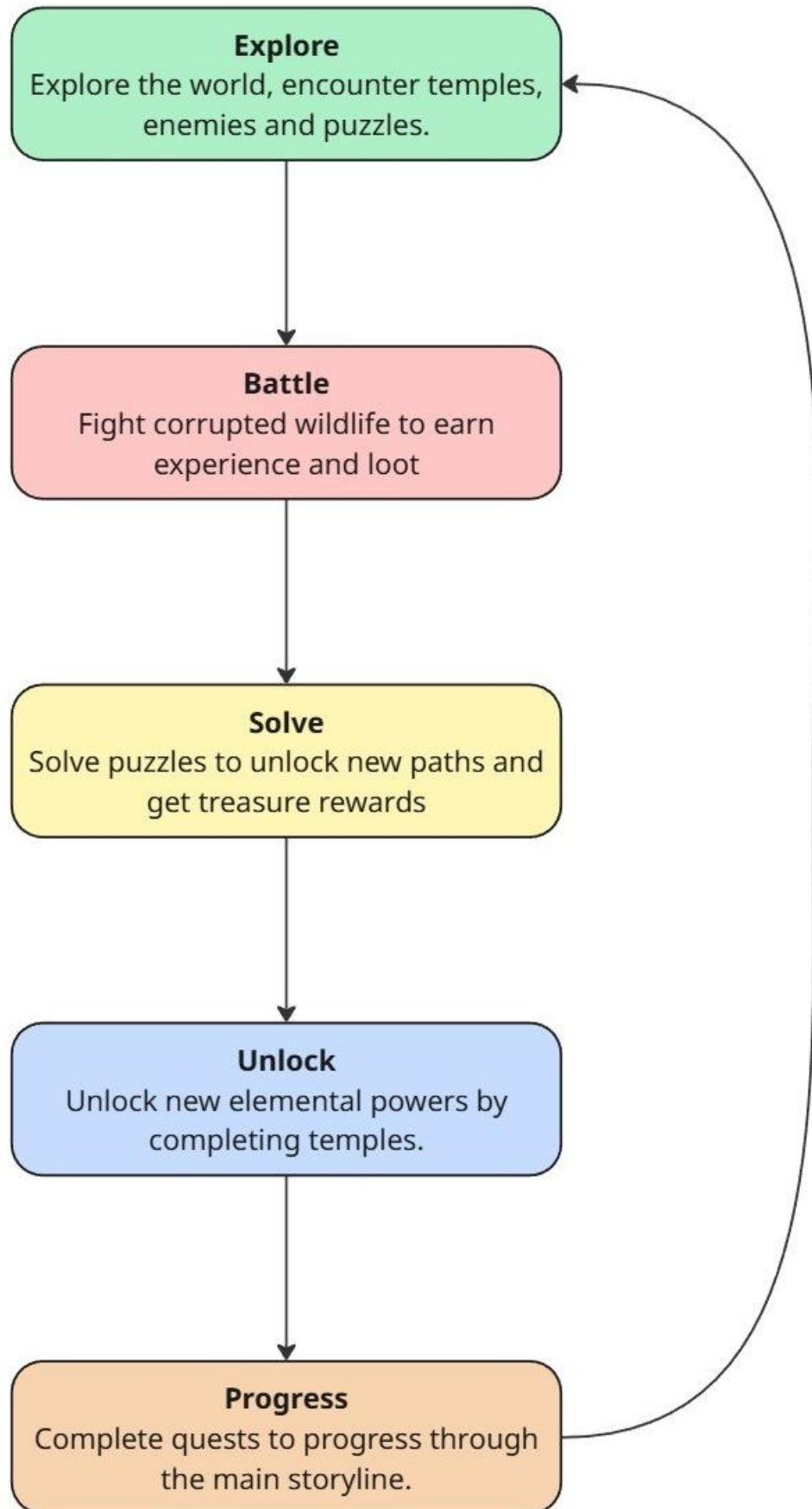
Between battles the player will come across puzzles which may need to be solved to advance. These puzzles can involve pushing blocks, activating switches or using elemental powers. Solving puzzles will unlock new paths or treasure. Some puzzles may require using the hamster companion to access small areas that the main character can't get to.

4. Unlock

Completing the procedurally generated temples will unlock new elemental powers for the player. These powers will allow the player to progress through previously blocked paths by doing things such as burning vines with fire or freezing water to create a traversable path across.

5. Progress

Using new powers and experience gained through combat the player can advance through the main questline, completing missions and exploring new areas. Side quests and optional puzzles can provide the player with additional loot. As the player levels up and equips more powerful gear their stats will increase allowing them to take on stronger enemies and more difficult temples.



(main gameplay loop – made in Miro)

5. Key Features

Core Features

- **Procedurally generated temples**
Elemental temples are procedurally generated ensuring a unique temple every time which is good for replay-ability.
- **Elemental powers**
Players unlock elemental powers such as fire, wind or ice by completing temples. These powers can be used in combat to give extra damage against enemies that have weakness to those types of damage. The powers will also aid in exploration to unlock new paths by doing things such as burning vines that are blocking the way.
- **Turn based combat**
Players will have to fight many of creatures who reside in the forest on their journey to the source of the corruption. This will be a turn-based system where the enemies will have reactive AI to respond to what the player does.
- **Dialogue and quests**
The dialogue system will allow players to interact with the characters around them and receive quests. Quests will guide the player along the main story and there will be optional side quests that the player can earn rewards by completing.

Secondary Features

- **Inventory and equipment system**
The inventory system will allow players to equip gear and weapons on their playable characters. Equipment will provide stat boosts like defence for armour and more damage with higher tier weapons.
- **Map and fast travel**
There will be a mini map in the corner of the screen, and a full screen map can be viewed in the menu. The player will be able to fast travel to previously visited locations to avoid having to make long journeys.
- **Dual gameplay perspectives**
The gameplay will be different depending on the current character the player is controlling. The main game will be played as a top-down game, but the player will be able to access side view platformer sections by using their pet hamster.
- **Companion**
The protagonist will be accompanied by their pet hamster. The player will be able to use their pet hamster to access small spaces which can contain loot or other key items needed to progress the main story.
- **Shops**
Shops and merchants will be spread throughout the game giving the player a place to spend the gold they earned and buy new equipment and consumables.

Stretch Features

- **Multiple party members**

More characters can be recruited to the team and be used in combat. They will have different damage types and be skilled in different types of weapons to each other allowing the player to have several damage types in their party to target weaknesses in enemies.

- **Language options**

If there is time, I would like to make the game available in French as well as English, to enable more people to play it.

- **Arcade mode**

Arcade mode will be playable from the start menu once the player has completed the main game. In arcade mode the player will be able to replay temples and try to get a high score by completing the most temples in a row without dying.

- **Environmental puzzles**

Optional puzzles will allow the player to gain treasure if they can solve them. Puzzles can include things such as push-able blocks or timed activations of things.

6. Look & Feel

Visual Style

The Mist will use a 2D top-down pixel art style for the main game world, paired with side-view pixel art for the hamster companion platforming sections. Tilesets and sprites will be hand drawn with inspiration coming from other 2D games such as *Sea of Stars*, *Hyper Light Drifter* and *Celeste*.

- **Environment**

The visual design of the environment will emphasize the contrast between the vibrant, healthy areas of the forest and the dark, corrupted zones. Clear visual cues will highlight interactable objects, hazards, and collectibles.

- **Sprites**

Elemental powers will be visually distinct, with fire, ice, and wind effects using bright, contrasting colours to stand out against the environment. Temples, traps, and natural landmarks will be carefully designed to guide players naturally through each area.

- **Animations**

Animations for characters and creatures will be simple but expressive, with corrupted wildlife showing visual indicators such as glowing eyes, and colour changes to signal danger.

Audio Style / Mood

The audio design will focus on creating an immersive player experience, enhancing both exploration and combat.

- **Music**

The music will change dynamically with locations and events. Calm, peaceful music in safe areas, transitioning to tense or ominous music when encountering enemies or entering heavily corrupted areas of the map.

- **Sound effects**

Creature sounds will depend on their corruption with corrupted animals making unsettling noises and regular animals being cute. Elemental powers will have distinct sound effects when used. Sounds such as footsteps and flowing water will help to create an immersive experience.

- **Mood**

Overall, the audio and visual styles aim to create a sense of curiosity and tension. Players should feel encouraged to explore with subtle visual and audio cues guiding them towards points of interest.

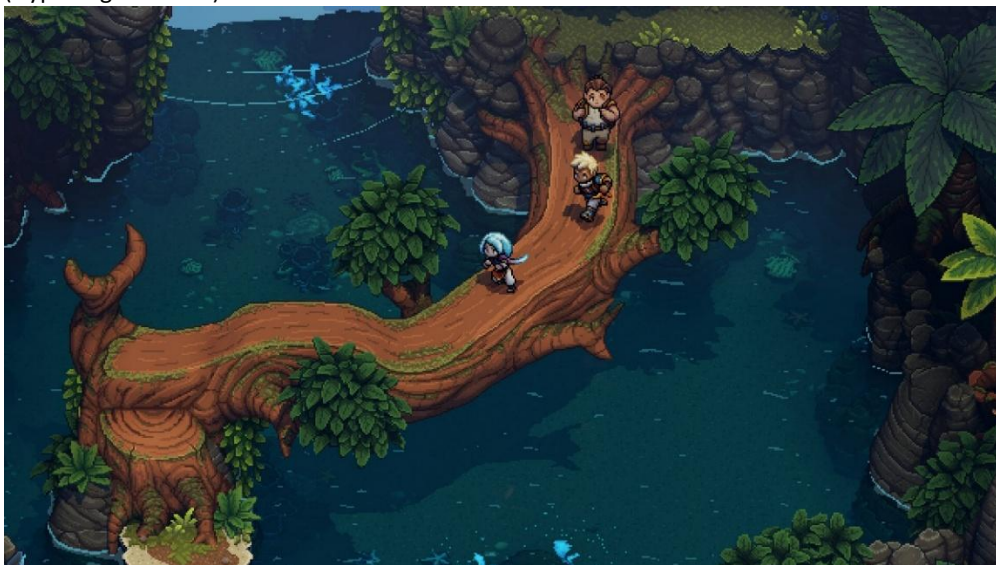
Reference Images



(Celeste)



(Hyper Light Drifter)



(Sea of Stars)

7. Narrative / Theme / Aesthetics

Story

A dark mist has begun spreading through the forest, corrupting the local wildlife and decaying the once vibrant landscape. The player sets out on an adventure aiming to reach the heart of the corruption and defeat the evil rat responsible for it. Accompanied by their pet hamster the player must explore the corrupted forest, complete puzzles and battle through temples to unlock elemental powers that will aid in the quest to restore peace to the lands.

Setting

- **The Ancient Forest:** A once peaceful woodland now overtaken by a dark corruption.
- **Elemental Temples:** Hidden structures tied to ancient fire, ice, and wind powers.
- **Corrupted Zones:** Dark, dangerous regions where the mist is strongest.
- **Hamster Platforming Realms:** Smaller side-view sections representing tight spaces only the hamster can navigate.

Characters

- **Eryn:** The protagonist. Seeks to destroy the source of corruption.
- **Milo:** Loyal pet hamster. Fits into small areas to help Eryn progress.
- **Nox:** The evil rat responsible for spreading corruption throughout the forest.

Tone

Mysterious and atmospheric. A balance between peaceful exploration and tense encounters.

Themes

- **Friendship and companionship:** The strong emotional bond between the player and their pet.
- **Restoration and healing:** Removing the corruption and returning life to the forest.
- **Courage in the face of darkness:** Facing corrupted creatures, entering dangerous temples and exploring deep into heavily corrupted areas.

Aesthetics

- **Mechanics**
Exploring, cleansing corruption, unlocking elemental powers, solving puzzles and switching to the hamster for platforming sections.
- **Dynamics**
World restoration, environmental storytelling, ability driven progression.
- **Aesthetics**
 - **Discovery:** Finding hidden paths and treasure.
 - **Fantasy:** Wielding elemental powers on the environment and in combat.
 - **Challenge:** Engaging in turn-based combat and solving complex puzzles.
 - **Narrative:** Uncovering the source of the corruption.

8. Technical Approach

Game Engine

Unity will be used to develop the game. Unity is well suited for 2D games and offers built in tools for tilemaps, animations, physics and UI. These features allow for rapid prototyping. Unity's tilemap system will be used to build modular rooms quickly, enabling the procedural generation of temples with varied layouts and puzzles. Unity is the game engine that I have the most experience in as it is what we used for the majority of our game development modules in university, making it the obvious choice.

Programming Language

C# will be used for all the scripting. C# integrates seamlessly with Unity, supports object-oriented programming and is widely used in professional game development.

Tools

- **Aseprite**
Aseprite will be used to create pixel art, animations and Tilesets for both the top-down and side-scrolling sections of the game. Aseprite is an excellent tool for pixel art allowing easy editing of sprites and creation of animations.
- **Rider**
Rider will be used for C# code development and debugging. It will help to maintain clean, error free code throughout development.

Data Storage

Game data will be stored locally using a package imported into Unity from Bayat Games. The package allows serializable data to be saved and loaded in JSON files, providing reliable persistence.

Version Control

GitHub will be used to manage all source code, assets and project updates. Regular commits will correspond with sprint goals.

Deployment Platform

The game will be developed for Windows PCs, with keyboard and mouse as the primary input method. The game build will be published to itch.io for playtesting.

Key Systems

- **Procedural generation**

Temple layouts will be generated using Binary Space Partitioning (BSP). Each temple space is recursively divided into smaller regions, with modular room prefabs placed within these partitions. Rooms contain predefined puzzles, enemy spawn points, and chest locations, and are connected by corridors to ensure full navigability. Constraints will ensure the temple is solvable and paths and exits aren't blocked.

- **Reactive AI**

Enemies will inherit from a base enemy class. Each enemy will have defined stats, abilities and weaknesses. A finite state machine will determine the actions on each turn based on the current situation to react appropriately to the players moves. Reactive decisions will be made using a priority scoring such as choosing to attack if the player is almost dead rather than healing itself if its health is low.

- **Puzzles**

Core and optional puzzles will interact with elemental powers, environmental objects and the hamster companion. Puzzle logic will use event-driven programming to detect when the player does things such as triggering the water to freeze when the player uses their ice powers while in range of and facing in the direction of water.

- **Inventory**

The inventory system will store the players items, equipment and gold. Items will be created as scriptable objects, allowing more objects to be easily added into the game. The inventory UI will be displayed using Unity's canvas system displaying items with tooltips and allowing the player to manage their items and equipment.

- **Dialogue**

The dialogue system will use a custom dialogue tool built as a Unity Editor window. This allows the visual creation of branching dialogue trees without needing to modify code. Conversations are stored in scriptable objects with nodes representing lines of dialogue and options. Each node stores the text and speaker. New nodes can be created, connected, edited and deleted using the editor window. The system evaluates conditions such as quest progression or having a quest item to determine which dialogue paths are displayed.

9. Literature & Research Summary

In this section I will be reviewing literature that will inform the design of *The Mist*. I will be covering procedural generation for my temples to give the game more replayability, exploring how algorithmic content creation can create unique layouts and challenges for each playthrough while maintaining a coherent gameplay experience. Procedurally generated temples will also need to integrate puzzles and environmental challenges in a way that feels natural and balanced for the player.

Another important research topic is AI, as I want my enemies to react to what the player does. I aim to learn how reactive and adaptive AI systems can make turn-based combat engaging and strategic without feeling unfair or predictable. Understanding AI behaviour will also help me design encounters that complement puzzle-solving and exploration, ensuring that combat remains challenging but fair.

Puzzle design in games will be a key focus to ensure that the puzzles are not only challenging but also intuitive, logical, and rewarding to solve, encouraging exploration and problem solving. These puzzles will interact with the procedural layouts and AI systems, allowing player choices in combat and exploration to influence puzzle opportunities and outcomes.

2D top-down RPGs often blend exploration, narrative, and combat with light puzzle-solving. Recent trends also show heavy use of procedural generation and adaptive behaviour systems to enhance replayability and engagement, providing opportunities for designers to create dynamic, responsive experiences that feel fresh on each playthrough. Through this literature review, I aim to draw insights from existing research to inform the core design systems of *The Mist*, including puzzle mechanics, temple layouts, and enemy AI behaviour, ensuring that the game offers a cohesive and engaging player experience.

Puzzle Design in Games

Puzzle design is central to engagement in adventure and RPG games. Puzzles are a great way to create interactive game experiences. The foundations of modern puzzle design can be traced to the earliest computer games. *Colossal Cave Adventure* created by William Crowther in 1976 introduced the first interactive digital logic puzzles and established exploration-based puzzle solving as a core game structure (Digital Humanities Quarterly, 2007).

Adam Bishop (2010), in *Puzzles Should Have Clues*, emphasizes that players must be provided with sufficient information to solve puzzles logically. Without clear clues, players rely on trial-and-error rather than reasoning, which can break immersion and reduce the sense of fairness. Bishop suggests using environmental signifiers, visible mechanics, and consistent logic to communicate the rules of each puzzle, allowing players to anticipate outcomes and develop strategies.

Josh Bycer (2018), in *A Study into Puzzle Design*, distinguishes between puzzles based on internal logic versus those reliant on designer logic. Internal logic puzzles, which adhere to the consistent rules of the game world, are generally more satisfying and replayable, while designer logic puzzles can feel arbitrary and frustrating. Bycer also highlights the importance of structured difficulty curves, where puzzles gradually increase in complexity as the player masters underlying mechanics.

Damien Allan (2017), in *Designing Video Game Puzzles*, argues that the most compelling puzzles are simple at their core but flexible in application. Allan recommends analogue mechanics (systems that allow a range of states or behaviours) over purely binary solutions (things like levers or locked doors), as they provide greater opportunities for emergent problem-solving. He also stresses the value of prototyping and iterative testing, allowing designers to detect unintended solutions or player workarounds and incorporate them into the puzzle's design.

From these sources, I learned that puzzles must be readable, fair, and gradually complex to maintain player engagement. Clear communication and internal logic are crucial to avoid frustration, and analogue mechanics allow for emergent solutions that enhance replayability. These insights directly influenced *The Mist's* design. Elemental powers and hamster mechanics are structured to teach the player, scale in difficulty, and encourage creative problem solving throughout the game.

Reactive and Adaptive AI in Games

Reactive AI refers to systems where non-player characters (NPCs) or enemies respond immediately to player actions or environmental stimuli. Unlike adaptive AI, which adjusts difficulty over time, reactive AI relies on predefined rules, state machines, or behaviour trees to produce intelligent, moment-to-moment responses. This allows enemies to appear smart and responsive without needing long-term planning.

Garg (2025), in *A Developer's Guide to Reactive, Deliberative, and Hybrid AI Agents*, explains that reactive agents function in the bottom layer of AI architectures, handling immediate responses such as "if attacked, dodge" or "if low health, retreat." This reactive layer can be combined with a deliberative layer for long-term planning, but even on its own, it creates responsive and engaging enemy behaviour.

Seest (2025), in *How to Design Games with Reactive Mechanics Using AI*, emphasizes the use of perception-action loops, where NPCs continuously sense the player's position or actions and immediately decide on countermoves. This approach makes the game feel more immersive and "alive," as enemies react naturally to player behaviour in real time.

Fuertes (2024) discusses reactive agent architectures, where agents map sensor inputs directly to actions without deliberation. Event-driven architectures like this allow AI to respond quickly to changes in the environment, making them especially useful for tactical or turn-based combat scenarios.

While reactive AI governs immediate responses, Singh (2024), in *How Adaptive AI in Video Games is Shaping the Future of Gaming*, highlights that some modern games combine reactive and adaptive elements. Adaptive AI observes player performance over time and adjusts enemy strength or encounter frequency to maintain engagement. Incorporating a small adaptive layer alongside reactive mechanics can keep players challenged without making encounters feel unfair.

From these sources, I learned that reactive AI is ideal for tactical enemy behaviour in turn-based RPG combat, allowing enemies to evaluate the player's party, anticipate actions, and respond intelligently on their turn. In *The Mist*, this informs the design of combat encounters: enemies could choose to target characters based on elemental weaknesses, use defensive or healing abilities in response to the player's moves, and coordinate attacks strategically. Singh's insights on adaptive AI also suggest that subtle adjustments to enemy stats or AI decision priorities over multiple encounters could complement reactive systems, maintaining balanced difficulty and flow throughout the game.

Procedural Generation

Procedural generation is the algorithmic creation of game content, such as levels, items, terrain, or entire dungeons, rather than manually designing each element. This approach not only saves development time but also increases replayability, creating unique experiences for players in each session.

One of the earliest uses of procedural generation in games was *Beneath Apple Manor* (1978), which randomly generated dungeon layouts, providing different level structures for each playthrough. However, *Rogue* (1980) is widely recognized as the first major game to fully embrace procedural generation, producing dynamic dungeon levels, items, and enemy placement. This system led to the creation of the “roguelike” genre and demonstrated how procedural generation could dramatically increase replayability (Compton, 2018).

Annand (2024) notes that early procedural generation was often driven by technical constraints, such as limited memory. Developers used algorithms to generate content on the fly, enabling expansive environments without requiring large amounts of storage. Games like *Rogue* and *Elite* became iconic examples of how procedural generation could create engaging, unpredictable gameplay experiences while conserving resources.

Autodesk (n.d.) emphasizes that procedural generation is not just a technical shortcut but a design philosophy. Modern games use procedural generation for terrain, levels, and even narrative structures, offering players fresh experiences on each playthrough. Procedural generation can also complement handcrafted design by filling optional areas or side dungeons with dynamically generated content, enhancing replayability while maintaining balanced core gameplay.

A commonly used technique for procedural level generation, particularly in dungeon-based games, is Binary Space Partitioning (BSP). BSP works by recursively dividing a large space into smaller subregions, which can then be populated with rooms, corridors, and gameplay elements. This structured approach allows designers to maintain control over level flow, room size, and spatial relationships while still benefiting from procedural variation. BSP is especially effective for creating readable, navigable layouts that avoid overlapping rooms and ensure logical connectivity.

For *The Mist*, BSP is well suited to the procedural generation of temples. The algorithm enables the creation of distinct room-based structures while preserving intentional pacing and spatial hierarchy. Key rooms, such as puzzle chambers or progression-critical areas, can be handcrafted and integrated within the BSP-generated layout, ensuring consistent challenge and narrative intent. Meanwhile, auxiliary rooms and pathways can be procedurally generated, allowing each temple to feel unique across playthroughs without compromising design integrity.

From these sources, I learned that procedural generation combines technical necessity and creative opportunity to enable high variability, save development resources, and introduce replayable experiences. For *The Mist*, employing BSP-based procedural generation allows each playthrough to feel unique while integrating handcrafted puzzles and ensuring consistent challenge.

Synthesis

Reviewing the literature on puzzle design, procedural content generation, and reactive/adaptive AI highlights several gaps and opportunities in existing game design research. Many studies on puzzle design focus on creating fair, internally logical challenges, but few explore how puzzles can be integrated seamlessly with dynamic game worlds or companion mechanics, like the hamster in *The Mist*. Similarly, procedural generation is often discussed in terms of technical efficiency and replayability, yet research rarely addresses how procedural systems can be combined with handcrafted puzzles and narrative pacing to maintain consistent player engagement. Finally, while reactive and adaptive AI techniques are well-documented, most examples emphasize either real-time or adaptive systems in isolation, leaving less guidance on how to combine reactive tactical decision-making with adaptive adjustments in a turn-based RPG context.

The Mist addresses these gaps by blending these systems into a unified experience: puzzles are designed to teach mechanics progressively and interact with both the player and companion, procedurally generated temples provide unique layouts while supporting handcrafted puzzle placement, and enemy AI uses reactive decision-making complemented by subtle adaptive scaling to maintain challenge in turn-based combat. Overall, the research inspired a design direction that balances structure and variability, ensuring engagement, replayability, and meaningful player choices throughout the game.

10. Scope and Feasibility

This project will deliver a top-down 2D RPG that combines exploration, puzzle-solving, and turn-based combat. The game is designed to be ambitious enough to demonstrate complexity and innovation, while remaining achievable within the available time and resources.

Scope

- **Core gameplay**
Players will explore a corrupted forest, solve puzzles, and battle enemies in a turn-based system. Combat will feature AI that reacts to the player's actions.
- **Procedural generation and elemental powers**
Completing procedurally generated temples will unlock elemental powers required to access new areas and overcome environmental obstacles, such as burning vines or freezing water.
- **Progression**
Players earn experience from combat and gain skill points to improve their stats. Optional side quests allow recruitment of additional party members, providing varied combat strategies.
- **Inventory and equipment**
Players will manage inventory, equip gear, and use items in combat, including healing or reviving party members. Gold can be spent in shops located throughout the world.
- **Puzzles and exploration**
There will be puzzles for the player to solve, some mandatory to progress through the main story and some more challenging optional ones that reward the player with loot. The game will encourage exploration with hidden paths and interactive environmental features.
- **Narrative interaction**
A dialogue system will allow players to interact with NPCs and objects to engage with the story. Players will receive quests by talking to NPCs. Non-essential NPCs will also have dialogue, to enhance immersion.

Feasibility

- The game is designed to be completed in approximately 3 hours, providing a full experience of exploration, combat, and puzzle-solving without overextending development scope.
- Stretch features such as multiple elemental powers, additional party members, extensive side quests, and full procedural temple variation may be reduced or simplified to focus on core systems. Fully polished art and audio will be replaced with placeholder assets during initial development.

This scope ensures that the project is large enough to demonstrate complexity and engagement but small enough to be completed to a high standard, prioritizing core gameplay and player experience.

11. Production Plan

Methodology

The development of the project will be managed follow a scrum-lite methodology. Scrum-lite is a version of scrum that is used in smaller teams or solo developers. The development will be split into sprints, each lasting 2 weeks. At the end of each sprint the progress is reviewed, and the next sprint can be planned based on what is the next priority. I will be using Trello to plan my sprints. This method should enable me to maintain a consistent production pace to meet the vertical slice deadline.

Tools

- **Trello**
Trello will be used to plan sprints and stay organised and on track with the production process. Tasks will be categorised into *Backlog* for tasks that I have fallen behind on, *Sprint planning* for tasks that need to be done in future sprints, *Current sprint* for the tasks I need to do in the current 2 week sprint, *In progress* for the tasks that are currently being worked on and *Done* for the completed tasks.
- **Miro**
Miro will be used to create the diagrams used to show things such as the core gameplay loop to be included in this document.
- **GitHub**
Used for version control to track changes to the Unity project and provide a backup of all development work. Work will be committed regularly to align with sprint goals.

Risk Analysis

- **Scope**
There is a risk of having too many things planned to add into the game and not having time to do it all. The stretch features of the game may have to be cut in order to prioritise the core and secondary game features.
- **Time management**
While I would like to create all the game art myself so that it all has the same style, I may have to resort to using online resources if I don't have the time.
- **Procedural generation**
I have planned to make each of the elemental temples have a different theme and mechanic such as a jungle temple containing traps or an ice temple containing puzzles. Creating all the differently styled rooms for this would take a huge amount of time so I might not be able to do this, and the temples will have to have the same general theme.

Sprint Schedule

Sprint	Dates	Goals
1	25/9/25 - 9/10/25	• Setup shared Word doc • Setup Trello board • Document section 10 - Scope and feasibility.
2	9/10/25 - 23/10/25	•
3	23/10/25 - 06/11/25	• Document sections 1,2,3,4,5 and 11 • Setup Miro board
4	06/11/25 - 20/11/25	• Document section 9 - Literature and Research • First playable prototype
5	20/11/25 - 04/12/25	• Document sections 6,7,8 and 12
6	04/12/25 - 18/12/25	• Dialogue and quests system • Procedural generation implementation • UI Menus • Audio Manager
7	18/12/25 – 01/01/26	• Combat System • Polish document • Xp and levelling system

Testing plan

Testing will take place at the end of each sprint with the current working prototype. This will ensure that each newly implemented system is tested and can be improved upon in the next sprint. Any bugs will be found early on which will make it easier to fix.

I will be thoroughly testing the game myself throughout the development. At the end of each sprint, I will get friends and family to test it for me to provide feedback. Feedback will be taken into consideration for future development and changes to existing systems.

All of the game's key features and mechanics will undergo testing.

12. Vertical Slice Specification

The vertical slice represents a minimal but functional implementation of *The Mist's* core systems, demonstrating exploration, combat, procedural generation, and progression. While not all planned features are included, the vertical slice focuses on validating the technical foundations and overall gameplay feel.

Playable Content

- **Procedurally Generated Temple**

One temple generated at runtime using a Binary Space Partitioning (BSP) algorithm. The temple is constructed from rooms placed within BSP partitions and connected by corridors, ensuring a readable layout with a clear start and exit.

- **Enemy**

One enemy type is included, engaging the player in turn-based combat and demonstrating reactive AI behaviour that responds to player actions.

- **Elemental power**

One elemental power will be available, given to the player when they complete the temple. The power can be used to unlock paths.

- **Levelling**

Players will be able to increase their stats using skill points gained from levelling up. Experience will be granted to the player on defeating an enemy.

- **Dialogue**

One NPC is available for interaction, showcasing the dialogue system and narrative delivery.

Functional Systems

- Top-down player movement and interaction. Sprint key.
- Turn-based combat with reactive enemy AI.
- BSP-based procedural temple generation.
- Dialogue system with branching and player choices.
- Experience, levelling, and stat progression.

Temporary Assets

The vertical slice uses placeholder or early-stage assets, including prototype pixel art, temporary UI elements, and non-final audio. These assets support gameplay clarity but do not represent final presentation quality.

Win Condition

Defeating the enemy within the procedurally generated temple.

Lose Condition

The player's health reaching zero during combat, triggering a game over state.

Known Exclusions

The following features are not included in the vertical slice due to time constraints:

- Environmental and elemental puzzles.
- Hamster companion gameplay and platforming sections.
- Multiple enemy types and elemental powers.
- Inventory management UI beyond core systems.
- Polished art, animations, and audio

Vertical Slice Summary

This vertical slice validates the core design pillars of *The Mist* by demonstrating BSP-based procedural generation, turn-based combat with reactive AI, and player progression within a cohesive gameplay loop. While puzzle mechanics are not present, the slice establishes a solid technical foundation for future development and expansion.

References

Jerz, D. (2007) *Somewhere Nearby is Colossal Cave: Examining Will Crowther's Original "Adventure" in Code and in Kentucky*. Digital Humanities Quarterly. Available at: <https://dhq.digitalhumanities.org/vol/1/2/000009/000009.html>

Bishop, A. (2010) *Puzzles Should Have Clues*. Game Developer. Available at: <https://www.gamedeveloper.com/design/puzzles-should-have-clues>

Bycer, J. (2018) *A Study into Puzzle Design*. Game Developer. Available at: <https://www.gamedeveloper.com/design/a-study-into-puzzle-design>

Allan, D. (2017) *Designing Video Game Puzzles*. Game Developer. Available at: <https://www.gamedeveloper.com/design/designing-video-game-puzzles>

Compton, C. (2018) *Going Rogue-like: When to Use Procedurally Generated Environments in Games*. Rempton Games. Available at: <https://remptongames.com/2018/01/25/going-rogue-like-when-to-use-procedurally-generated-environments-in-games/>

Annand, G. (2024) *By Design: Procedural Generation*. SuperJump Magazine. Available at: <https://www.superjumpmagazine.com/by-design-procedural-generation/>

Autodesk (n.d.) *Procedural Generation*. Available at: <https://www.autodesk.com/solutions/media-entertainment/procedural-generation>

Singh, H. (2024) *How Adaptive AI in Video Games is Shaping the Future of Gaming*. Blocktunix. Available at: <https://blocktunix.com/adaptive-ai-in-video-games-shaping-future/>

Garg, J. (2025) *A Developer's Guide to Reactive, Deliberative, and Hybrid AI Agents*. GoCodeo. Available at: <https://www.gocodeo.com/post/a-developers-guide-to-reactive-deliberative-and-hybrid-ai-agents>

Seest, T. (2025) *How to Design Games with Reactive Mechanics Using AI*. HowDoYouGame. Available at: <https://howdoyougame.com/how-to-design-games-with-reactive-mechanics-using-ai/>

Fuertes, R. A. (2024) *Reactive Agent Architectures: How Simple AI Models Respond to Dynamic Environments*. SmythOS. Available at: <https://smythos.com/developers/agent-development/reactive-agent-architectures/>

Credits

Credits: List all third-party assets used in your game. Group them according to type:

Images

Models

Animations

Audio

Original music and SFX created for the game by Ashley Holloway

Cameras

Player controllers

Appendix (Optional)

Include diagrams, flowcharts, mockups, or reference images if helpful.